9 Modular Exponentiation and Cryptography

9.1 Modular Exponentiation

Modular arithmetic is used in cryptography. In particular, **modular exponentiation** is the cornerstone of what is called the RSA system.

We consider first an algorithm for calculating modular powers. The **modular exponentiation** problem is:

compute $g^A \bmod n$, given g, A, and n.

The obvious algorithm to compute $g^A \bmod n$ multiplies g together A times. But there is a much faster algorithm to calculate $g^A \bmod n$, which uses at most $2\log_2 A$ multiplications.

The algorithm uses the fact that one can reduce modulo n at each and every point. For example, that $ab \bmod n = (a \bmod n) \times (b \bmod n) \bmod n$. But the key savings is the insight that g^{2B} is the square of g^B.

```
DEXPO(g,A,n)
  if A = 0 then return 1
  else if A odd {
      z = dexpo(g, A − 1, n)
      return(zg mod n)        % uses g^A = g × g^(A−1)
      }
  else {
      z = dexpo (g, A/2, n)
      return(z^2 mod n)       % uses g^A = (g^(A/2))^2
      }
```

Note that the values of g and n are constant throughout the recursion. Further, at least every second turn the value of A is even and therefore is halved. Therefore the depth of recursion is at most $2\log_2 A$.

We can do a modular exponentiation calculation by hand, by working out the sequence of values of A, and then calculating $g^A \bmod n$ for each of the A, starting with the smallest (which is $g^0 = 1$).

EXAMPLE 9.1. *Calculate* $3^{12} \bmod 5$.

A	$g^A \bmod n$
12	$4^2 \bmod 5 = 1$
6	$2^2 \bmod 5 = 4$
3	$3 \times 4 \bmod 5 = 2$
2	$3^2 \bmod 5 = 4$
1	$3 \times 1 \bmod 5 = 3$
0	1

► **For you to do!** ◄

1. *Use the* DEXPO *algorithm to calculate* $4^{14} \bmod 11$.

9.2 Modular Exponentiation Theorems

We start with a famous theorem called **Fermat's Little Theorem**.

Theorem 9.1 *Fermat's little theorem. If p is a prime, then for a with $1 \leq a \leq p-1$,*

$$a^{p-1} \bmod p = 1.$$

PROOF. Let S be the set $\{\, ia \bmod p : 1 \leq i \leq p-1 \,\}$. That is, multiply a by all integers in the range 1 to $p-1$ and write down the remainders when each is divided by p. Actually, we already looked at this set: it is the row corresponding to a from the multiplication table for p. And in Theorem 8.1 we showed that these values are all distinct. Therefore, S is actually just the set of integers from 1 up to $p-1$.

Now, let A be the product of the elements in S. To avoid ugly formulas, we use $x \equiv_p y$ to mean $x \bmod p = y \bmod p$. And we use Π-notation, which is the same as Σ-notation except that it is the product rather than the sum. By Theorem 8.1 and the above discussion,

$$\prod_{i=1}^{p-1} ((ia) \bmod p) = \prod_{i=1}^{p-1} i$$

But, we can also factor out the a's:

$$\prod_{i=1}^{p-1} (ia) \bmod p \equiv_p a^{p-1} \prod_{i=1}^{p-1} i$$

It follows that

$$\prod_{i=1}^{p-1} i \ \equiv_p \ a^{p-1} \prod_{i=1}^{p-1} i$$

Divide both sides by $\prod_{i-1}^{p-1} i$ and we get that $a^{p-1} \ \equiv_p \ 1$; that is, $a^{p-1} \bmod p = 1$. $\Diamond$

The above result is generalized by **Euler's Theorem**. We will need the following special case later.

Theorem 9.2 *Special case of Euler's theorem. If a and $n = pq$ are relatively prime, with p and q distinct primes, then $a^{\phi} \bmod n = 1$ where $\phi = (p-1)(q-1)$.*

We omit the proof.

9.3 Encryption and Decryption

Encryption is used to send messages secretly. The sender has a message or **plaintext**. **Encryption** by the sender takes the plaintext and a **key** and produces **ciphertext**. **Decryption** by the receiver takes the ciphertext and a key and produces the plaintext. Ideally, encryption and decryption with the key should be easy, without the key it should be hard.

A famous idea is the **Caesar cipher**. Here the plaintext is written in letters. Then to encrypt it, we choose some small positive integer k, and replaces each letter by the letter k places along in the alphabet, with wrap around. For example, if the plaintext is the phrase `WAYNE RULES` and $k = 4$, then the ciphertext is `AECRI VYPIW` where spaces are left unchanged in this example.

In traditional cryptography, the encryption key has to be kept secret, since it is essentially the same as the decryption key. But in **public-key cryptography**, the two keys are related but in some way that is very difficult to get one from the other. So you can publish the one key and keep secret the other key. Indeed, this has been commercially exploited by RSA cryptosystems, formed by Rivest, Shamir, and Adleman.

9.4 RSA Described

Here is the original RSA cryptosystem. This allows Alice to send messages secretly to Bob. (Tradition requires that it is Alice and Bob.) That is, the ciphertext is unintelligible to anyone else.

1. Bob randomly picks two large distinct primes p and q.
2. Bob calculates $n = pq$ and $\phi = (p-1)(q-1)$.
3. Bob randomly picks some integer a with $1 \leq a < \phi$ such that a and ϕ are relatively prime.
4. Bob publishes n and a, and the public encryption function is $e(x) = x^a \bmod n$. That is, to send message M, Alice calculates and sends $M^a \bmod n$.
5. Bob uses the Extended Euclidean Algorithm to calculate $b = a^{-1}$ modulo ϕ.
6. The decryption function is $d(y) = y^b \bmod n$. That is, to decrypt message N, Bob calculates $N^b \bmod n$.

EXAMPLE 9.2. *A baby example.*

Take $p = 7$ and $q = 11$. Then $n = 77$ and $\phi(n) = 6 \times 10 = 60$. We need an a less than 60 that is relatively prime to 60; say $a = 13$. By the Extended Euclidean Algorithm (or a handy-dandy computer), $b = 37$.

If $M = 26$, then Alice calculates $y = 26^{13} \bmod 77 = 75$. Bob calculates $x = 75^{37} \bmod 77$, which equals 26.

We now show that RSA really works as a cryptosystem.

Theorem 9.3 *For all $M \in \mathbb{Z}_n$, $d(e(M)) = M$.*

PROOF. Well, let me first assume that M and n are relatively prime. Then, using the notation $x \equiv_n y$ to mean $x \bmod n = y \bmod n$,

$$\begin{aligned} d(e(M)) &\equiv_n (M^a)^b \\ &= M^{c\phi+1} \qquad \text{for some integer } c \\ &= (M^\phi)^c M \\ &\equiv_n M \qquad \text{by Theorem 9.2.} \end{aligned}$$

It can also be shown that the claim is true if M and n do have a common factor. But actually, in that case the cryptosystem has been broken, since then we have found a factor of n. $\Diamond$

Solutions to Practice Exercises

1.

A	$g^A \bmod n$
14	3
7	5
6	4
3	9
2	5
1	4
0	1

Exercises

9.1. (a) Compute $2^{38} \bmod 7$.

(b) Compute $3^{29} \bmod 20$.

(c) Compute $5^{33} \bmod 13$.

9.2. (a) Prove that $(a+b)^p \bmod p = (a^p + b^p) \bmod p$ if p is a prime.

(b) Use part (a) to give a proof of Fermat's Little Theorem.

9.3. Using the Binomial Theorem (and without using Fermat's Little Theorem), prove that for any odd prime p, it holds that $2^p \bmod p = 2$.

9.4. Take $p = 5$, $q = 11$ and $a = 27$ in RSA. Verify that $b = 3$. If $M = 4$, what does Alice send? If Bob receives 9, what was M?

9.5. Take $p = 11$, $q = 13$ and $a = 37$ in RSA. Verify that $b = 13$. If $M = 62$, what does Alice send?

9.6. Take $n = 65$ and $a = 11$ in RSA. Determine b. If $M = 2$, determine what Alice sends.